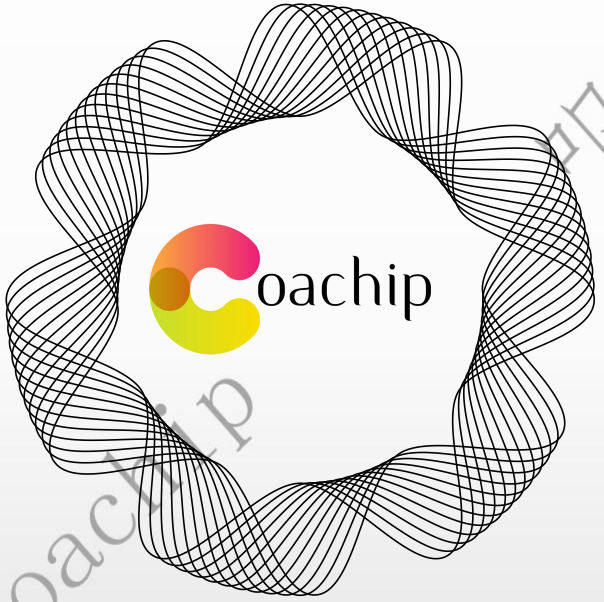


SV多线程之间的通信

叩特电子Coachip

课程内容及目标

课程内容：

- 线程的使用
- 线程间通信的需求
- event（事件）
- semaphore（旗语）
- mailbox（信箱）

目标：

- 了解event、semaphore和mailbox的用法

叩特电子Coachip

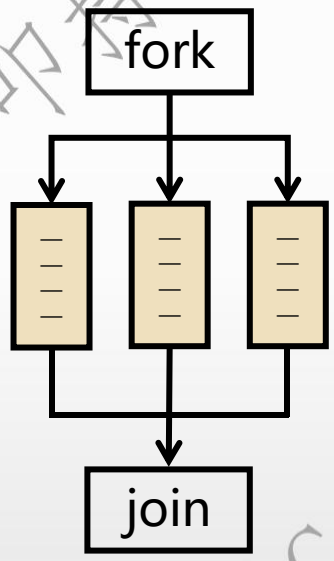
线程的使用

- Verilog中对语句分组使用：
  - begin...end（顺序执行）
  - fork...join（并发执行）
- System Verilog中引入两种新的创建线程的方法：
  - fork...join_any
  - fork...join_none

叩特电子Coachip

fork...join（代码并发执行）

必须等内部所有语句执行完成才能继续块内后续的线程。




```
1  module test();
2      initial begin
3          $display("%t:AAAA",$time);
4          #10 $display("%t:BBBB",$time);
5
6          fork
7              $display("%t:CCCC",$time);
8              #50 $display("%t:DDDD",$time);
9              #10 $display("%t:EEEE",$time);
10             begin
11                 #30 $display("%t:FFFF",$time);
12                 #10 $display("%t:GGGG",$time);
13             end
14         join
15         $display("%t:HHHH",$time);
16         #80 $display("%t:IIII",$time);
17     end
18
19 endmodule
```

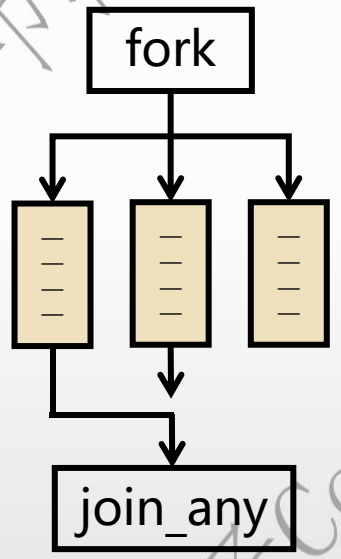
仿真结果：

```
  0:AAAA
 10:BBBB
 10:CCCC
 20:EEEE
 40:FFFF
 50:GGGG
 60:DDDD
 60:HHHH
140:IIII
```

叩特电子Coachip

fork...join_any

fork...join_any对块内语句进行调度，当第一个语句完成后，父线程继续执行。




```
1  module test();
2      initial begin
3          $display("%t:AAAA",$time);
4          #10 $display("%t:BBBB",$time);
5
6          fork
7              $display("%t:CCCC",$time);
8              #50 $display("%t:DDDD",$time);
9              #10 $display("%t:EEEE",$time);
10             begin
11                 #30 $display("%t:FFFF",$time);
12                 #10 $display("%t:GGGG",$time);
13             end
14         join_any
15         $display("%t:HHHH",$time);
16         #80 $display("%t:IIII",$time);
17     end
18
19 endmodule
```

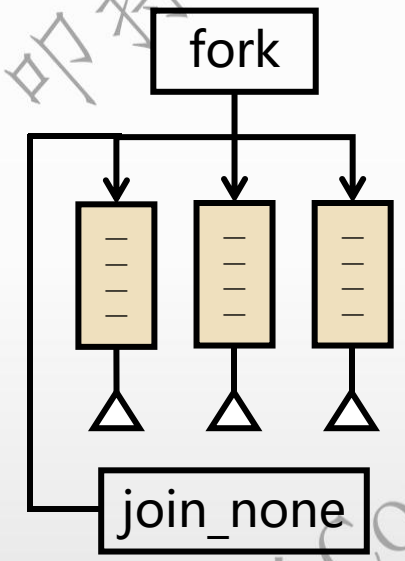
仿真结果：

```
 0:AAAA
10:BBBB
10:CCCC
10:HHHH
20:EEEE
40:FFFF
50:GGGG
60:DDDD
90:IIII
```

叩特电子Coachip

fork...join_none

fork...join_none在调度其块内语句时，父线程继续执行。




```
1  module test();
2      initial begin
3          $display("%t:AAAA",$time);
4          #10 $display("%t:BBBB",$time);
5
6          fork
7              $display("%t:CCCC",$time);
8              #50 $display("%t:DDDD",$time);
9              #10 $display("%t:EEEE",$time);
10             begin
11                 #30 $display("%t:FFFF",$time);
12                 #10 $display("%t:GGGG",$time);
13             end
14         join_none
15         $display("%t:HHHH",$time);
16         #80 $display("%t:IIII",$time);
17     end
18
19 endmodule
```

仿真结果：

```
 0:AAAA
10:BBBB
10:HHHH
10:CCCC
20:EEEE
40:FFFF
50:GGGG
60:DDDD
90:IIII
```

叩特电子Coachip

线程间通信的需求

我们可以将硬件模块内部的各个过程语句块（always块）理解为多个在0时刻就运行的线程（thread），这些线程是并发执行并且不会退出的。对于硬件的过程块，不同逻辑/时序块之间的通信或者同步，实际是通过信号的变化来完成的。例如各种握手信号req/ack, valid/ready等；

对软件而言，如果多个线程(多个task同时执行)，该如何对这些软件上的多个线程进行通信和同步呢？如果将多个对象也抽象为多个线程，那么多个对象之间要通信，又该如何实现呢？

叩特电子Coachip

线程间通信的需求

解决办法：

- event（事件）
- semaphore（旗语）
- mailbox（信箱）

叩特电子Coachip

event（事件）

event（事件）：事件是静态的同步对象句柄（可以像参数一样在子程序中传递），它用来同步多个并发的进程，比如某个进程等待着事件，而另一个进程则触发这个事件。

事件的触发和阻塞等待

触发： -> event_handle

等待：@ or wait(event_handle.triggered)

@操作符是阻塞的，也就是说过了操作的时间点，如果没有捕捉到，那么就会被阻塞在那里，直到下一次到来。如果没有下一次，就一直阻塞。

在SV中，引入了triggered属性，该属性可用于查询事件是否已经被触发，而不是只检测当前时刻，线程只等待该结果，而不是在@处阻塞。

叩特电子Coachip

event（事件）

事件的触发和阻塞等待

触发： -> event_handle

等待：@ or wait(event_handle.triggered)

特征：

- 可以被赋值给其它事件；这样两个事件变量（句柄）会指向同一个对象，触发任意一个变量就触发这个事件
- 可以传给队列，函数和任务

叩特电子Coachip

event（事件）

```
event eve;
event tmp;

initial begin
    tmp = eve;
    #10 -> tmp;
end

 initial begin
     wait(eve.triggered);
     $display("@%0t : event eve receive trigger",$time);
end
```

```
---------------------------------
@10 : event eve receive trigger    //执行结果
---------------------------------
```

叩特电子Coachip

event（事件）

```
program automatic test();

event e1,e2;
initial begin
    $display("@%0d:1:before trigger",$time);
    -> e1;
    @e2;
    $display("@%0d:1:after trigger",$time);
end

initial begin
    $display("@%0d:2:before trigger",$time);
    -> e2;
    @e1;
    $display("@%0d:2:after trigger",$time);
end

endprogram
```

因为触发是瞬态的，因此按先后顺序，只有@e2是可以触发的，另一种就没了，仿真结果如下：

```
@0:1:before trigger
@0:2:before trigger
@0:1:after trigger
```

如果调换两个initial块的顺序，则输出：

```
@0:2:before trigger
@0:1:before trigger
@0:2:after trigger
```

由此我们也可以知道timeslot中initial块是有执行顺序的，顺序为自上而下。

叩特电子Coachip

event（事件）

```
program automatic test();

event e1,e2;
initial begin
    $display("@%0d:1:before trigger",$time);
    -> e1;
    wait(e2.triggered);
    $display("@%0d:1:after trigger",$time);
end

initial begin
    $display("@%0d:2:before trigger",$time);
    -> e2;
    wait(e1.triggered);
    $display("@%0d:2:after trigger",$time);
end

endprogram
```

因为e1先触发，因此wait(e1.triggered)先实现，先打印2再打印1，因此仿真结果如下：

```
@0:1:before trigger
@0:2:before trigger
@0:2:after trigger
@0:1:after trigger
```

叩特电子Coachip

event（事件）

➢wait_order的使用

```
module tb;
  event a, b, c;
  initial begin
    #10 -> a;
    #10 -> b;
    #10 -> c;
  end
  initial begin
    wait_order (a,b,c)
        $display ("Events were executed in the correct order");
    else
        $display ("Events were NOT executed in the correct order !");
  end
endmodule
```

wait_order阻塞等待多个事件的触发，并且要求这多个事件按照用户决定顺序触发。wait_order可以和else一同使用，当多个事件按顺序触发时，执行wait_order后的语句，否则执行else后的语句。

```
--------------------------------------------------
Events were executed in the correct order //执行结果
--------------------------------------------------
```

叩特电子Coachip

mailbox（信箱）

Mailbox 是一种在进程之间交换消息的机制。数据可以通过一个进程发送到Mailbox，然后由另一个进程获取。数据可以是任何有效的SystemVerilog数据类型，包括类class数据类型。

No.	Method	Description
1	new()	Creat a mailbox
2	put()	Place a message in a mailbox
3	try_put()	Try to place a message in a mailbox without blocking
4	get() / peek()	Retrieve a message from a mailbox
5	num()	Returns the number of messages in the mailbox
6	try_get() / try_peek()	Try to retrieve a message from a mailbox without blocking

叩芯电子Coachip

mailbox（信箱）

```
class transaction;
  rand bit valid;
  rand bit [7:0] data;
endclass
```

```
class generator;
        mailbox #(transaction) gen2drv;
        transaction tr;
        function new (input mailbox #(transaction) gen2drv);
            this.gen2drv=gen2drv;
        endfunction
        task gen(input int num);
           for(int i=0;i<num;i++) begin
              tr=new();
               assert(tr.randomize)
               gen2drv.put(tr);
               $display("trans valid =%d",tr.valid);
               $display("trans data =%d",tr.data);
               #1ns;
           end
         endtask
endclass
```

叩特电子Coachip

mailbox（信箱）

```
class driver;
  mailbox #(transaction) gen2drv;
  transaction tr;
  function new (input mailbox #(transaction) gen2drv);
     this.gen2drv=gen2drv;
  endfunction
  task run;
     tr=new();
     while(1) begin
        gen2drv.get(tr);
        $display("trans valid =%d",tr.valid);
        $display("trans data =%d",tr.data);
     end
   endtask
endclass
```

```
program test;
  mailbox #(transaction) gen2drv;
  driver drv;
  generator gen;
  initial begin
     gen2drv=new();
     drv=new(gen2drv);
     gen=new(gen2drv);
     fork
        gen.gen(5);
        drv.run;
     join_any
   end
endprogram
```

叩特电子Coachip

semaphore（旗语）

Semaphore: 使用旗语可以实现对同一资源的访问控制。当测试平台中存在一个资源，如一条总线，对应着多个请求方，而实际物理设计中只允许单一驱动时，便可使用旗语。

在system verilog中一个线程如果请求“钥匙”而得不到，则会一直阻塞，多个阻塞的线程会以先进先出的方式进行排队

semaphore基本操作函数:

```
new();   //Create a semaphore with a specified number of keys
get();   //Obtain one or more keys from the bucket
put();   //Return one or more keys into the bucket
try_get(); //Try to obtain one or more keys without blocking
```

即特电子Coachip

semaphore（旗语）

```
class bridge;
  semaphore key;
  function new ();
     this.key=new(1);
  endfunction
  task go_on(string name);
    $display("@%0t: %s wants to go trough the brdige",$time,name);
    key.get();
    #1ns;
    $display("@%0t: %s gets permission to go trough the brdige",$time,name);
    #10ns;
   endtask
  task go_off(string name);
    $display("@%0t: %s goes off the brdige",$time,name);
    key.put();
    #1ns;
    $display("@%0t: %s returned the key.",$time,name);
   endtask
endclass
```

叩特电子Coachip

semaphore（旗语）

```
program test;
    bridge brg;
    string name1=”Alex”;
    string name2=”Bob”;
    string name3=”Calvin”;
    string name4=”Denise”;
    initial begin
        brg=new();
        fork
            begin
                brg.go_on(name1);
                brg.go_off(name1);
            end
            begin
                brg.go_on(name2);
                brg.go_off(name2);
            end
            begin
                brg.go_on(name3);
                brg.go_off(name3);
            end
            begin
                brg.go_on(name4);
                brg.go_off(name4);
            end
        join
    end
endprogram
```

叩特电子Coachip

Q&A

昂特电子Coachip